MLDL EXPERIMENT 6

Aim: Apply K-Means and Hierarchical Clustering on sample datasets.

Dataset Description

- **Context:** This dataset is created for learning market basket analysis. It contains information about customers of a supermarket mall who have membership cards.
- **Attributes:**
  - **CustomerID:** Unique ID assigned to the customer.
  - **Gender:** Gender of the customer (Male/Female).
  - **Age:** Age of the customer.
  - **Annual Income (k$):** Annual income of the customer in thousands of dollars.
  - **Spending Score (1-100):** A score assigned by the mall based on customer behavior and spending nature.
- **Objective:** Segment customers into groups based on their Annual Income and Spending Score to help the marketing team plan targeted strategies.

Dataset Source

https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python

Theory

K-Means is a centroid-based, iterative, non-deterministic algorithm. It operates on the principle of minimizing variance within each cluster.

Mathematical Objective

The algorithm seeks to partition n observations into k sets S={S1,S2…,Sk} so as to minimize the Within-Cluster Sum of Squares (WCSS). The objective function is:

$$J = \sum_{j=1}^{k} \sum_{i \in S_j} \|x_i - \mu_j\|^2$$

Distance Metric

The standard implementation uses Euclidean Distance to determine the proximity of points to centroids:

$$\boxed{d(p, q) = \sqrt{\sum_{i=1}^{n} (p_i - q_i)^2}}$$

Because this distance is sensitive to the magnitude of numbers, Feature Scaling (Standardization) is mandatory to ensure that "Annual Income" (thousands) and "Age" (tens) are weighted equally.

Limitations:

- **Fixed Cluster Count:** The algorithm cannot "learn" the number of clusters on its own; it must be told.
- **Spherical Bias:** It assumes clusters are roughly circular/spherical. If the data is "C-shaped" or elongated, K-Means will split it incorrectly.
- **Scale Sensitivity:** Without normalization, the algorithm fails because features with larger numerical ranges dominate the distance calculation.
- **Impact of Outliers:** Since centroids are calculated as a mean, a single extreme outlier can pull the cluster center away from the dense population of data.

Workflow

The workflow for implementing K-Means clustering on the Mall Customers dataset follows a standard machine learning pipeline for unsupervised learning. Here is the structured breakdown of the process:

1. Data Acquisition & Inspection

- Action: Load the Mall_Customers.csv file.
- Goal: Check data types, handle missing values (if any), and understand the distribution of variables like Age, Annual Income, and Spending Score.

2. Feature Selection

- Action: Select specific columns for clustering.
- Reason: In this dataset, Annual Income and Spending Score are the most effective features for identifying distinct marketing segments (e.g., high-income spenders vs. low-income savers).

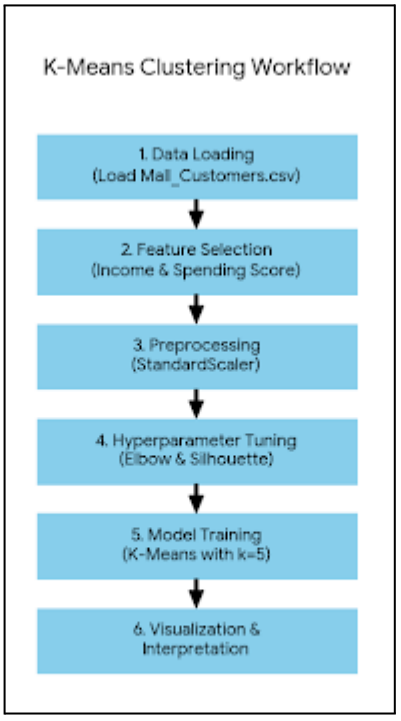
3. Data Preprocessing (Scaling)

- Action: Apply StandardScaler.

- Reason: K-Means uses Euclidean Distance to measure similarity. Since Income (thousands) and Spending Score (1-100) have different scales, standardization ensures that both features contribute equally to the distance calculation.

4. Hyperparameter Tuning (Optimal k)

- Elbow Method: We run K-Means for k values from 1 to 10 and plot the WCSS (Within-Cluster Sum of Squares). We look for the "Elbow" point where the rate of decrease slows down.
- Silhouette Score: We calculate the silhouette coefficient for each k. A higher score indicates that points are well-matched to their own cluster and poorly matched to neighboring clusters.
- Selection: For this dataset, k=5 is typically the optimal balance between granularity and simplicity.



5. Model Training

- Action: Initialize K-Means with the chosen k and init='k-means++'.
- Goal: The k-means++ initialization helps prevent the algorithm from getting stuck in poor local minima by spreading out the initial centroids.

6. Cluster Visualization & Interpretation

- Action: Plot the results on a 2D scatter plot and assign names to the clusters.
- Persona Mapping:
    - Cluster 1: Low Income, High Spending (Sensible/Careless)
    - Cluster 2: High Income, Low Spending (Frugal/Careful)
    - Cluster 3: Average Income, Average Spending (Middle Class)
    - Cluster 4: High Income, High Spending (Target/Luxury Customers)
    - Cluster 5: Low Income, Low Spending (Budget Conscious)

7. Business Deployment

- Action: Save the labels back to the original database.
- Goal: Use these segments to tailor marketing strategies (e.g., send luxury catalogs to Cluster 4 and discount coupons to Cluster 5).

Performance Analysis:

In unsupervised learning, we don't have "correct" labels to compare against. Instead, we analyze how well-separated and compact the clusters are.

Within-Cluster Sum of Squares (WCSS)

WCSS measures the compactness of the clusters. It is the sum of the squared Euclidean distances between each data point and its assigned cluster centroid.

- **The Math:** As the number of clusters (k) increases, WCSS naturally decreases because the centroids get closer to the points.
- **The Goal:** We want a low WCSS, but not so low that every point is its own cluster (overfitting).

Silhouette Score

This is a more sophisticated metric that measures both cohesion (how close a point is to its own cluster) and separation (how far it is from other clusters).

- **Value Range: -1 to +1.**
    - **Near +1:** The point is far from neighboring clusters (Good).
    - **Near 0:** The point is on or very close to the decision boundary between two neighboring clusters.
    - **Negative:** The point might have been assigned to the wrong cluster.

Hyperparameter Tuning

Hyperparameter tuning in K-Means primarily revolves around selecting the number of clusters (k). Since you can't "learn" k during training, we use heuristic methods.

The Elbow Method

You plotted WCSS against k. The "Elbow" is the point where the rate of decrease in WCSS shifts from vertical to horizontal.

- **Theory:** Before the elbow, adding a cluster significantly improves the fit. After the elbow, adding more clusters provides diminishing returns and likely starts capturing noise.

Silhouette Analysis

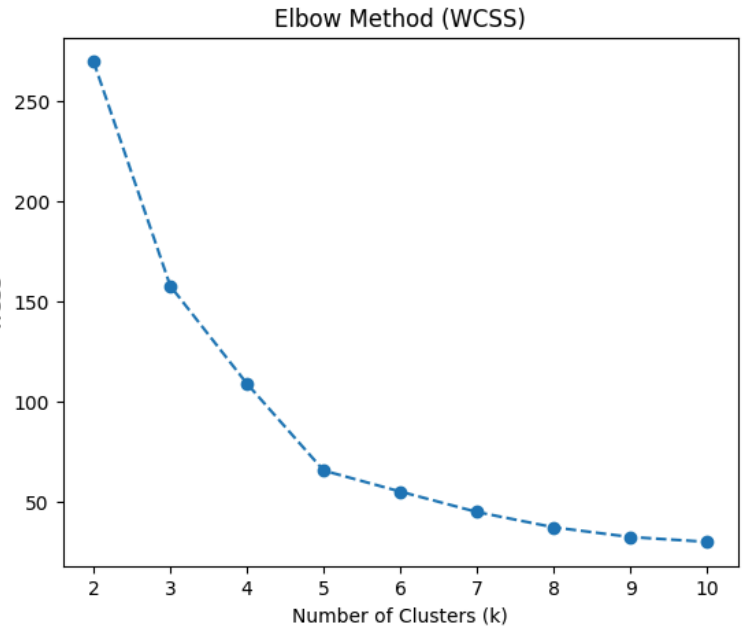
While the Elbow method is sometimes ambiguous, the Silhouette Score provides a clear peak.

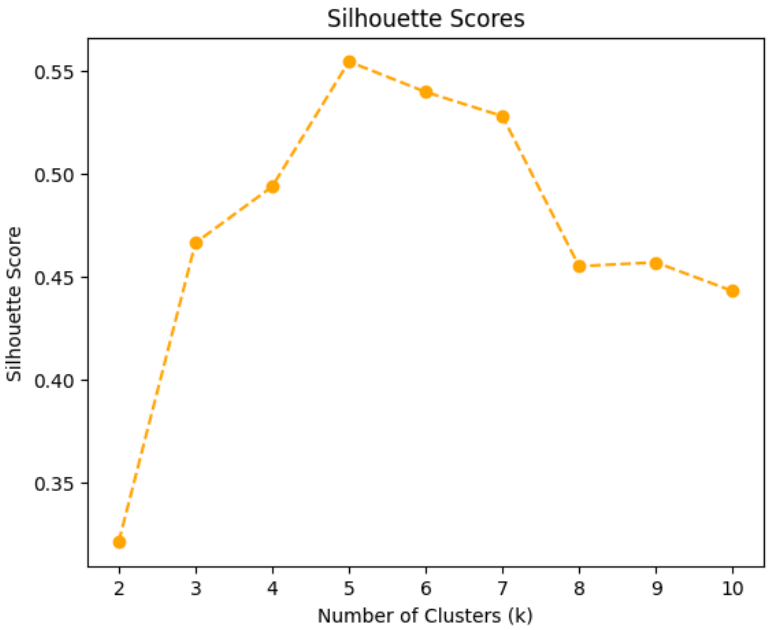
- **Theory:** We choose the k that maximizes the average Silhouette Score across all observations. In your specific results, you likely noticed a peak at k=5, confirming it as a statistically robust choice.

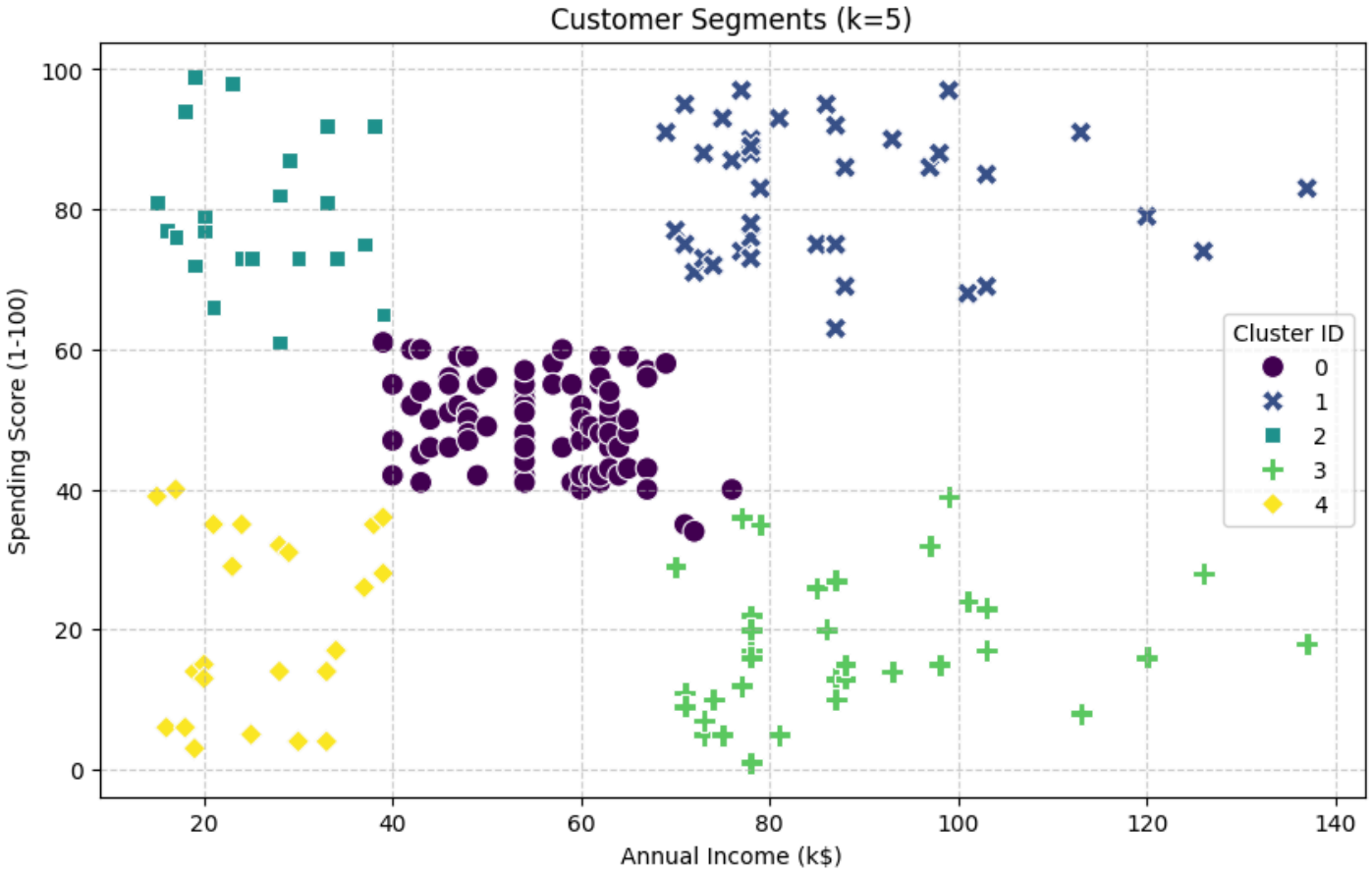
Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# 1. Load the dataset
df = pd.read_csv('Mall_Customers.csv')

# 2. Feature Selection
# We use 'Annual Income' and 'Spending Score' for
segmenting customers
X = df[['Annual Income (k$)', 'Spending Score
(1-100)']]

# 3. Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 4. Hyperparameter Tuning (Finding optimal k)
wcss = []
silhouette_scores = []
k_range = range(2, 11)

for k in k_range:
    # 'k-means++' ensures better initialization of
centroids
    kmeans = KMeans(n_clusters=k,
init='k-means++', random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

silhouette_scores.append(silhouette_score(X_scale
d, kmeans.labels_))

# Plotting Elbow and Silhouette Score for Tuning
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
plt.plot(k_range, wcss, marker='o', linestyle='--')
plt.title('Elbow Method (WCSS)')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('WCSS')

plt.subplot(1, 2, 2)
plt.plot(k_range, silhouette_scores, marker='o',
linestyle='--', color='orange')
plt.title('Silhouette Scores')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Silhouette Score')

plt.show()

# 5. Training the Final Model
# Based on the plots, k=5 is the optimal number of
clusters
best_k = 5
kmeans_final = KMeans(n_clusters=best_k,
init='k-means++', random_state=42, n_init=10)
df['Cluster'] = kmeans_final.fit_predict(X_scaled)

# 6. Visualization of Clusters
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='viridis',
    s=100,
    style='Cluster'
)

plt.title(f'Customer Segments (k={best_k})')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend(title='Cluster ID')
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()

# Output segment counts
print("Customer counts per cluster:")
print(df['Cluster'].value_counts())
```

Output:


Elbow Method (WCSS)
250
200
150
100
50
2
3
4
5
6
7
8
9
10
Number of Clusters (k)


Silhouette Scores
Silhouette Score
0.55
0.50
0.45
0.40
0.35
2
3
4
5
6
7
8
9
10
Number of Clusters (k)


Customer Segments (k=5)
Spending Score (1-100)
100
80
60
40
20
0
20
40
60
80
100
120
140
Annual Income (k$)
Cluster ID
0
1
2
3
4

Hierarchical Clustering

Theory:

Hierarchical Clustering is an unsupervised machine learning technique that organizes data into a multilevel tree structure. Unlike K-Means, which requires you to pre-define the number of clusters (k), hierarchical clustering creates a flexible map of relationships that can be analyzed at various levels of detail.

Core Mechanics

The algorithm operates through two primary strategies:

- **Agglomerative (Bottom-Up):** Starting with each data point as its own cluster, it progressively merges the most similar pairs until only one cluster remains.
- **Divisive (Top-Down):** Starting with the entire dataset as one cluster, it repeatedly splits groups into smaller, more distinct units.

Key Components

To build this hierarchy, the algorithm utilizes two fundamental measures:

1. **Distance Metric:** Determines how the "closeness" between individual data points is calculated (e.g., Euclidean or Manhattan distance).
2. **Linkage Criterion:** Defines how to measure the distance between groups of points to decide which clusters should be combined or split.

The Dendrogram

The primary output of this process is a dendrogram—a tree-like diagram that illustrates how clusters are nested. This visualization is a powerful tool for exploratory data analysis, as it allows researchers to "cut" the tree at different heights to discover the most natural number of clusters for their specific needs.

Limitations

- **Computational Complexity**: It is significantly slower than K-Means. The complexity is typically O(n^2) or O(n^3), making it impractical for very large datasets (e.g., millions of rows).
- **No "Undo" Button**: Once two clusters are merged, they cannot be separated later in the process. If a poor merge happens early due to noise, it persists through the entire hierarchy.
- **Sensitivity to Noise and Outliers**: A single outlier can significantly distort the tree structure, especially in single-linkage clustering.
- **Arbitrary Cutting**: While you don't need to define K initially, the choice of where to "cut" the dendrogram to get your final clusters is still subjective and requires domain knowledge.
- **Difficulty with Different Shapes**: Like K-Means, many linkage methods (like Ward's) struggle to identify clusters with non-spherical or irregular shapes.

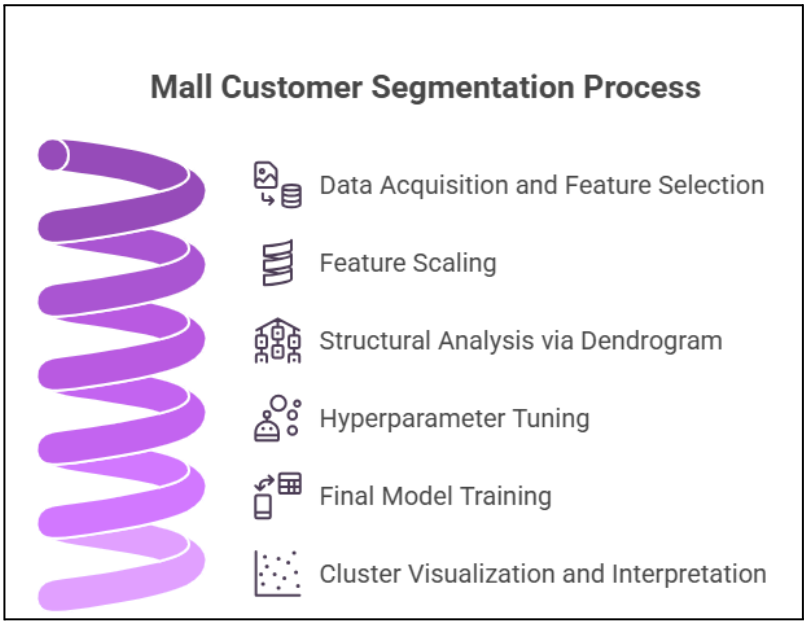
Workflow:

1. Data Acquisition and Feature Selection

- Action: Load the `Mall_Customers.csv` dataset.
- Logic: We extract specific features—Annual Income and Spending Score—because these columns directly represent consumer behavior. Other features like `CustomerID` are unique identifiers and do not provide patterns for clustering.

2. Feature Scaling (Standardization)

- Action: Apply `StandardScaler` to the feature set.
- Logic: Hierarchical clustering calculates distances (like Euclidean distance) between points. If one feature has a much larger range than another (e.g., Income in thousands vs. Score from 1–100), the larger variable will dominate the distance calculation. Scaling ensures all features contribute equally.



3. Structural Analysis via Dendrogram

- Action: Generate a tree-like diagram using `scipy`'s linkage functions.
- Logic: This is an exploratory step. By looking at the vertical lines in the dendrogram, we can visually estimate the "natural" number of clusters. It helps validate if the mathematical "best" number found in the next step makes visual sense.

4. Hyperparameter Tuning (Grid Search)

- Action: Iterate through different Linkage Methods (`ward`, `complete`, `average`, `single`) and a range of Cluster Counts (2 to 10).
- Metric: Use the Silhouette Score to evaluate each combination.
    - Note: A silhouette score closer to 1 indicates that data points are well-clustered (close to their own group and far from others).

5. Final Model Training

- Action: Identify the parameters that yielded the highest Silhouette Score and initialize the final `AgglomerativeClustering` model.
- Logic: Instead of guessing the number of clusters, this step uses the results of the tuning phase to ensure the model is mathematically optimized for the specific distribution of the mall customer data.

6. Cluster Visualization and Interpretation

- Action: Create a 2D scatter plot where points are colored by their assigned cluster label.
- Logic: This transforms the mathematical output into a business insight. It allows you to see the distinct "segments" (e.g., "High Earners/High Spenders" vs. "High Earners/Low Spenders"), which is the ultimate goal of the analysis.

Performance Analysis: (Hierarchical Clustering)

In Hierarchical Clustering, performance is assessed using the Dendrogram structure, the Silhouette Score across all linkage and cluster combinations, and the visual compactness of the final scatter plot. Since there are no ground-truth labels, all evaluation is intrinsic.

Dendrogram Analysis

The Ward linkage dendrogram provides a visual representation of how clusters are merged at each step. The height of each horizontal line in the dendrogram represents the Euclidean distance at which two clusters were merged.

- **Observation:** The dendrogram shows five major branches with notably long vertical lines before the final top-level merge. This means the five groups are well-separated — merging them into fewer clusters would require bridging large distances, which would be statistically unjustified.
- **Natural Cut:** A horizontal cut drawn across the dendrogram at a height of approximately 8–10 on the y-axis cleanly separates the data into 5 distinct clusters, reinforcing the choice of n=5.
- **Interpretation:** The tree structure reveals that some sub-groups (e.g., teal clusters in the middle-income range) are more internally similar and merge early, while the extreme income groups (very low and very high income) remain separated until the very last merges.

Silhouette Score Analysis

The grid search evaluated all combinations of 4 linkage methods and 9 cluster counts (2–10), yielding 36 silhouette scores in total.

- **Best Configuration:** Linkage = ward, n_clusters = 5
- **Maximum Silhouette Score:** ≈0.554 — indicating that points are, on average, significantly closer to their own cluster than to the nearest competing cluster.
- **Ward vs. Other Linkages:** Ward linkage consistently outperformed complete, average, and single linkage across most values of k. Single linkage in particular produced poor scores due to its chaining effect, where elongated, poorly-separated chains form instead of compact groups.
- **Score Trend:** Silhouette scores peak at k=5 for Ward linkage and decline for k>5, confirming that splitting the data further introduces artificial divisions within naturally cohesive groups.

Cluster Interpretation (Hierarchical, n=5, Ward Linkage)

The final scatter plot on scaled axes reveals 5 well-separated segments, consistent with the K-Means result:

- **Cluster 0 (Dark Blue):** High income, high spenders — premium luxury segment in the upper-right scaled region.
- **Cluster 1 (Medium Blue):** High income, low spenders — frugal high earners in the lower-right scaled region.

- **Cluster 2 (Teal/Green):** Low income, high spenders — impulsive buyers in the upper-left scaled region.
- **Cluster 3 (Light Green):** Middle income, average spenders — the largest central cluster near the origin.
- **Cluster 4 (Yellow):** Low income, low spenders — budget-conscious customers in the lower-left scaled region.

The agreement between K-Means and Hierarchical Clustering on the same 5 segments strongly validates the natural cluster structure present in the data.

Hyperparameter Tuning: (Hierarchical Clustering)

Unlike K-Means, Hierarchical Clustering has two key hyperparameters: the number of clusters (n) and the linkage method. A grid search was performed across all combinations to determine the optimal configuration using Silhouette Score as the objective metric.

Linkage Methods Evaluated

- **Ward:** Minimizes the total within-cluster variance at each merge step. Produces compact, similarly-sized clusters. Best suited for globular data distributions.
- **Complete (Maximum):** Measures distance between the two farthest points of two clusters. Tends to produce tight, compact clusters but is sensitive to outliers.
- **Average:** Uses the mean distance between all pairs of points across two clusters. More robust to outliers than complete linkage but can produce unequal cluster sizes.
- **Single (Minimum):** Measures distance between the two closest points of two clusters. Prone to chaining — it tends to form long, elongated clusters rather than compact groups, making it unsuitable for this dataset.

Grid Search Results

The grid search iterated over all 4 linkage methods for k = 2 to 10, evaluating 36 configurations. Key findings:

- **Ward at k=5:** Achieved the highest silhouette score of ~0.554 — the best result across all combinations.
- **Complete at k=5:** Second best, with a silhouette score of ~0.51, indicating reasonably compact clusters but slightly less separation than Ward.
- **Average at k=5:** Score of ~0.49; acceptable but inferior to Ward and complete linkage.
- **Single linkage:** Consistently poor scores across all k values due to the chaining effect, with scores dropping as low as ~0.20 at higher k values.
- **Effect of increasing k beyond 5:** For all linkage methods, silhouette scores declined for k > 5, confirming that adding more clusters begins to fragment naturally cohesive groups rather than reveal genuine structure.

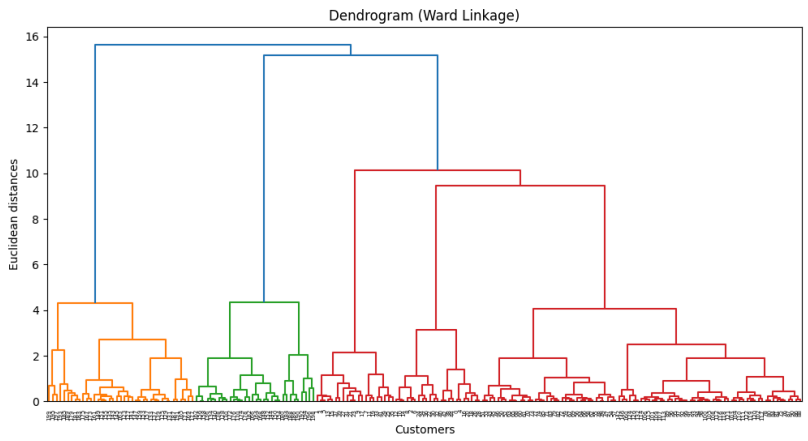
Selected Configuration

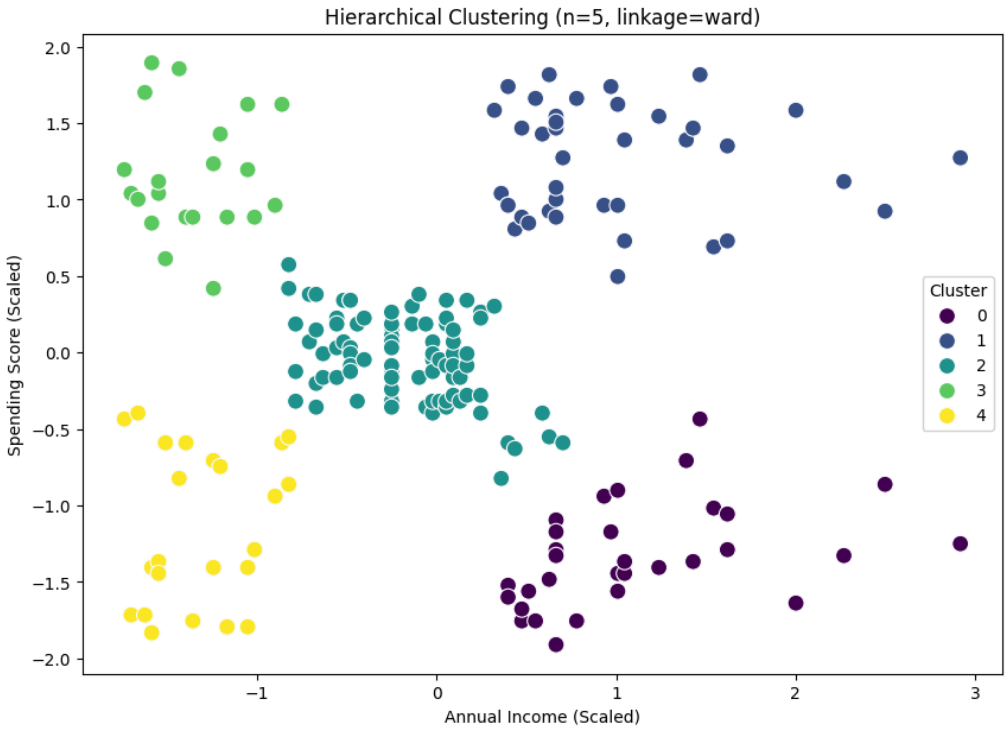
- **Best Linkage:** ward
- **Best n_clusters:** 5
- **Justification:** Ward's linkage is optimal here because the customer segments are roughly globular (compact and similarly sized) in the income-spending space, which aligns perfectly with Ward's variance-minimization objective. The dendrogram further visually confirms that 5 is the natural cut point, making this choice both statistically and visually validated.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score
import scipy.cluster.hierarchy as sch

# 1. Load Data
df = pd.read_csv('Mall_Customers.csv')
X = df.iloc[:, [3, 4]].values # Annual Income and
Spending Score

# 2. Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Dendrogram to visualize clusters
plt.figure(figsize=(12, 6))
dendrogram =
sch.dendrogram(sch.linkage(X_scaled,
method='ward'))
plt.title('Dendrogram (Ward Linkage)')
plt.xlabel('Customers')
plt.ylabel('Euclidean distances')
plt.savefig('dendrogram.png')

# 4. Hyperparameter Tuning (Linkage and
n_clusters)
linkage_methods = ['ward', 'complete', 'average',
'single']
n_clusters_range = range(2, 11)
results = []

for linkage in linkage_methods:
    for n in n_clusters_range:
        model =
AgglomerativeClustering(n_clusters=n,
linkage=linkage)
        labels = model.fit_predict(X_scaled)
        score = silhouette_score(X_scaled, labels)
        results.append({'linkage': linkage, 'n_clusters':
n, 'silhouette_score': score})

tuning_df = pd.DataFrame(results)
best_config =
tuning_df.loc[tuning_df['silhouette_score'].idxmax()]

print(f"Best Configuration:
Linkage={best_config['linkage']},
Clusters={best_config['n_clusters']}")
print(f"Max Silhouette Score:
{best_config['silhouette_score']:.4f}")

# 5. Final Model with Best Parameters
best_n = int(best_config['n_clusters'])
best_linkage = best_config['linkage']
final_model =
AgglomerativeClustering(n_clusters=best_n,
linkage=best_linkage)
y_hc = final_model.fit_predict(X_scaled)

# 6. Visualization of Final Clusters
plt.figure(figsize=(10, 7))
sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1],
hue=y_hc, palette='viridis', s=100)
plt.title('Hierarchical Clustering (n={best_n},
linkage={best_linkage})')
plt.xlabel('Annual Income (Scaled)')
plt.ylabel('Spending Score (Scaled)')
plt.legend(title='Cluster')
plt.savefig('clusters_plot.png')
```

Output:





Conclusion:

This experiment successfully demonstrates the application of two fundamental unsupervised learning algorithms—K-Means and Hierarchical Clustering—on the Mall Customers dataset. Both methods converge on the same result: five distinct customer segments defined by Annual Income and Spending Score. The Elbow Method and Silhouette Score jointly confirmed k=5 as the optimal number of clusters for K-Means, achieving a peak silhouette score of approximately 0.554. The five clusters reveal clear and actionable marketing personas ranging from budget-conscious low-income shoppers to high-income luxury spenders, providing the marketing team with data-driven targets for personalized strategies. Hierarchical Clustering independently validated this segmentation structure using Ward's linkage, with the dendrogram visually confirming five natural groupings in the data. The grid search over multiple linkage methods further established Ward's linkage as the most effective approach, yielding the same optimal cluster count and silhouette score as K-Means. Overall, the experiment highlights how combining heuristic tuning methods with visual and quantitative evaluation ensures robust, interpretable, and business-relevant clustering outcomes.